university of groningen

faculty of science and engineering

Self Balancing Motorbike using Funnel Controller

Niek Fikkers - S3278204
Swathi Krishnanunni - S5931266
Ammar Akhlaq Ahmed - S6113613

Abstract

This report investigates the application of funnel control for real-time stabilization of a self-balancing motorcycle equipped with a flywheel. The system's objective is to regulate the roll (tilt) angle of the motorcycle to asymptotically track a zero reference, corresponding to the upright equilibrium position. A model-free, nonlinear funnel controller is employed to guarantee that the tracking error remains strictly within a predefined, time-varying performance (the funnel), despite uncertainties in system dynamics and external disturbances. The proposed controller adapts without requiring explicit knowledge of the motorcycle's full dynamics, relying only on real-time measurements of the tilt angle. By using a feedback mechanism, where control effort rises as the error gets closer to the funnel boundary, the control mechanism guarantees the performance requirements. In this report we have formulated the theory required for the experimental knowledge, modeling of the system and experimental validation which proves the robustness and effectiveness of the funnel control Mechanism in making bounded-error convergence and maintaining stability.

Contents

1 Introduction **3**

2 System Overview **4**

3 Equations of motion **5**

4 Filter **7**

5 PD(P) Controller **9**

6 Funnel Controller **12**
6.1 Degree 2 Funnel 12
6.2 Tuning the Funnel boundaries 14
6.3 Simulink application of the Funnel 15
6.4 Problems faced with the funnel 16

7 Conclusion **17**

8 Future Scope **17**

9 Appendix **19**
9.1 Moments of inertia 19
9.2 DC motor torque vs rotational speed 19
9.3 Constants used in PD(P) controller 19
9.4 Controller and Plant 20
9.5 Internal structure of Funnel controller and PID 22

1 Introduction

When a two-wheeled vehicle is subjected to external forces, maintaining balance becomes a difficult control issue due to perturbations and nonlinear behavior. This project concentrates on the creation and implementation of a funnel Controller. A flywheel is used by the controller to maintain the stability of a self-balancing motorbike. The tilt angle is controlled by the motorcycle when it is stationary or moving slowly, allowing it to remain upright, through the torque produced by a rotating flywheel.

The funnel controller is different from conventional control strategies, which does not need the model of the system. An adaptive control approach that doesn't use a model and ensures that the tracking error falls within a given time-varying range. The boundary is referred to as the funnel. Because of this, it is especially appropriate for uncertain systems like where specific characteristics of our motorbike, such as moment of inertia, damping, or disruptions, may be hard to ascertain either measure or forecast precisely.

A traditional Proportional Derivative (PD) controller is used in addition to the funnel controller to ensure the motorcycle's stability. The PD controller is in charge of producing corrective inputs to the DC motor based on the measured tilt angle and its rate of change. The system maintains a steady torque by constantly monitoring the tilt angle and making adjustments as needed. The PD controller attempts to minimize both steady-state and transient errors by modifying the control input in accordance with the present error. Its simplicity, ease of implementation, and real-time response make it a easy option for early testing thus making the PD controller a benchmark for comparison.

The tilt angle (roll angle) of the motorcycle is the output of interest in this application, and the reference signal is representing the ideal vertical posture as zero. The tilt angle is maintained within the specified range by the funnel controller. A shrinking boundary with time that permits early transients but enforces high long-term performance. The theoretical understanding of the funnel control, a little bit of PID, modelling of self-balancing bike and the experimental findings are all covered in this report.

2 System Overview

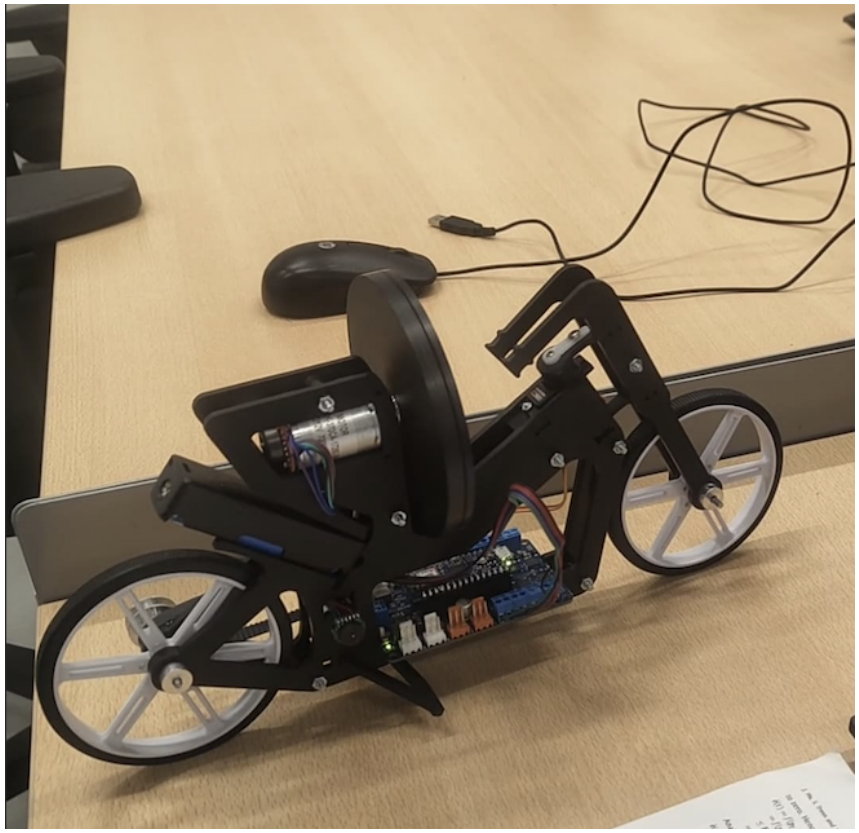

Figure 1: The bike that was used in this report to generate the data.

The primary objective is to regulate the motorcycle's tilt angle using a flywheel torque and maintain an upright posture under slight disturbances.The system consists of the following key components:

The flywheel acts as the balance correction actuator, makes up the flywheel mechanism. Using this torque, the system is effectively stabilised.

Inertial Measurement Unit (IMU) detects the motorbike's real-time tilt angle and its angular velocity.

The motor communicates with the flywheel motor to precisely regulate its rotational speed and torque output.

Servo Motor (Steering) improves the directional stability of the front wheel which is coupled to a servo motor.

3 Equations of motion

A PID or Funnel controller does not require the system dynamics to achieve its goal. However, the equations of motion will provide us with an insight into the dynamics of the system. Most importantly, it will provide us with the relative degree of the system. Namely, the Funnel controller implemented in Section 6 is for systems of relative degree 2.

To derive the equations of motion the bike is considered to be a pendulum with an attached flywheel. Consider Figure 2 in which the back of the bike can be seen. The variables θ and ω are defined, where the arrows for both denote the positive direction. The capital letters A,B,C and D denote in respective order, the axis of rotation for the pendulum, center of mass of the pendulum, center of mass and axis of rotation of the flywheel and lastly the end of the pendulum. The pendulum and the flywheel are considered to have a uniform and equal density.

In practice the point at which the bike is in balance is never exactly at $\theta = 0$ rad, but slightly deviated from this point. This amount is referred to as the bias in this report.

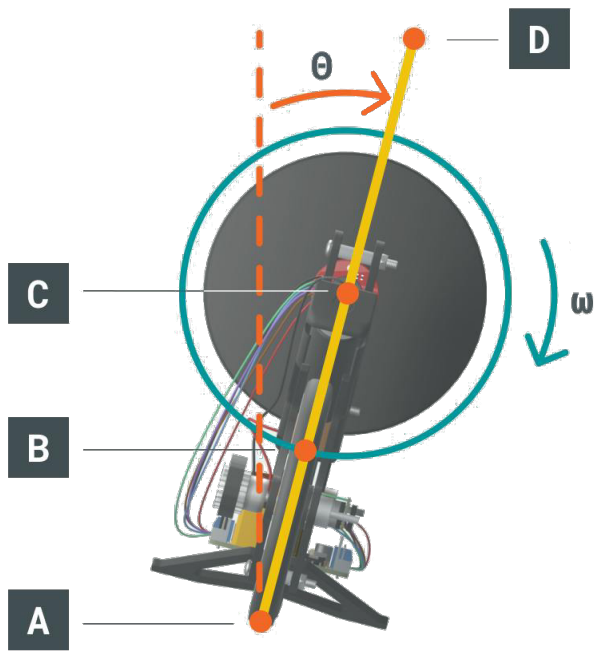

Figure 2: The back view of the bike with the definitions of the variables θ and ω. The capital letters A,B,C and D denote in respective order, the axis of rotation for the combined system, center of mass of the combined system, center of mass and axis of rotation of the flywheel and lastly the end of the pendulum. (This image has been corrected and the original comes from [3].)

Let's start with the equation describing the movement of the pendulum (rod). For this we use $\sum \tau = I\ddot{\theta}$,

$$\tau_r + \tau_f + \tau_m = (I_r^A + I_f^A)\ddot{\theta},$$

where τ_r is the torque coming from gravity acting on the pendulum (rod),
τ_f is the torque comes from gravity acting on the flywheel,
τ_m is the torque generated by the DC motor,
I_r^A is the moment of inertia of the pendulum (rod) when rotating around point A and
I_f^A is the moment of inertia of the flywheel when rotating around point A. The moments of inertia can be found in Appendix 9.1.

Secondly, we have the equation describing the movement of the flywheel.

$$\tau_m = I_f^C(\dot{\omega} + \ddot{\theta}),$$

where I_f^C is the moment of inertia of the flywheel when rotating around point C.

We can write these equations into the following state space model

$$\dot{x}(t) = \begin{bmatrix} \dot{\theta} \\ \ddot{\theta} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} \dot{\theta} \\ \frac{\tau_r + \tau_f + u(t)}{I_r^A + I_f^A} \\ \frac{u(t)}{I_f^C} - \ddot{\theta} \end{bmatrix} \tag{1}$$

$$y(t) = \theta(t), \tag{2}$$

with $u(t) = \tau_m$. The equations with all the terms explicitly determined can be found in Appendix 9.1. Observe that taking 2 time derivatives of the output $y(t)$ gives us an expression that contains the input $u(t)$. This is relevant for the funnel controller discussed in Section 8, as we have a system of relative degree 2 .

4 Filter

It is important to have good measurements coming from the IMU, as these will be used in the controller. In Figure 3, a graph of the measured $\dot{\theta}$ can be seen along with the filtered $\dot{\theta}$. This was done in an attempt to make the bike more stable and avoid spike in the inputs send to the DC motor. The time span in the graph is a rather short 1 s but this is because it takes less than approximately 0.5 s to go from $\theta = 0$ rad to $\theta = 0.24$ rad $\approx 14^o$. At this (arbitrarily chosen) angle the bike is considered fallen over in a safety mechanism. This angle is larger than the angle (exact angle is unknown) from which the bike can put itself back to $\theta = 0$.

However, no change in behaviour was noticed, but also no rigorous tests were done besides simply observing the bike balancing with and without the filter. This filter is used for all the tests described in this report.

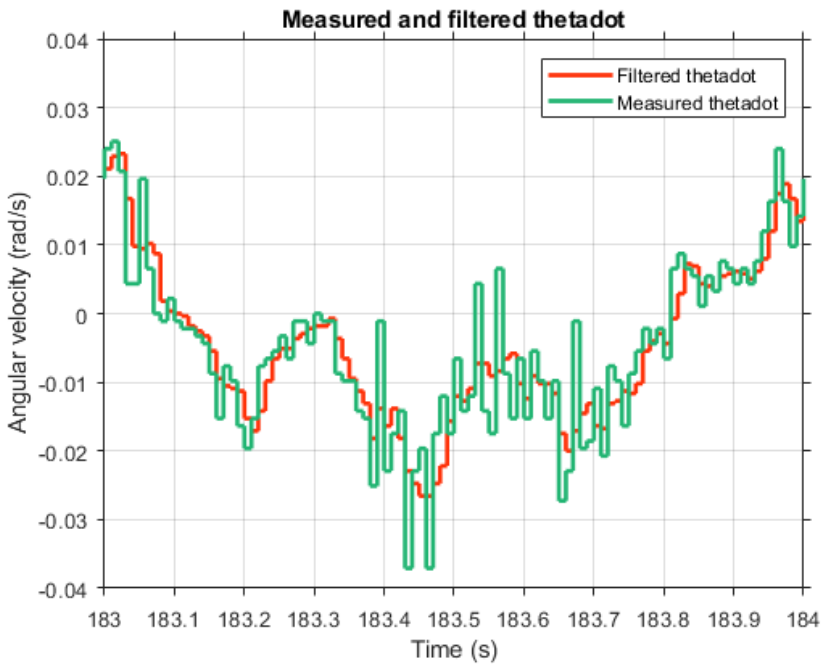

Figure 3: Filtering of $\dot{\theta}$, using a moving average of the last 3 values. During this period the bike was in balance using the PDP controller (Section 5) and had a flywheel speed of $\omega \approx 21$ rad/s.

In Figure 4 an extreme case is shown where the flywheel is spinning very fast (relatively speaking). This causes more and larger vibrations than observed in Figure 3, also notice that the scale of the y axis has been changed.

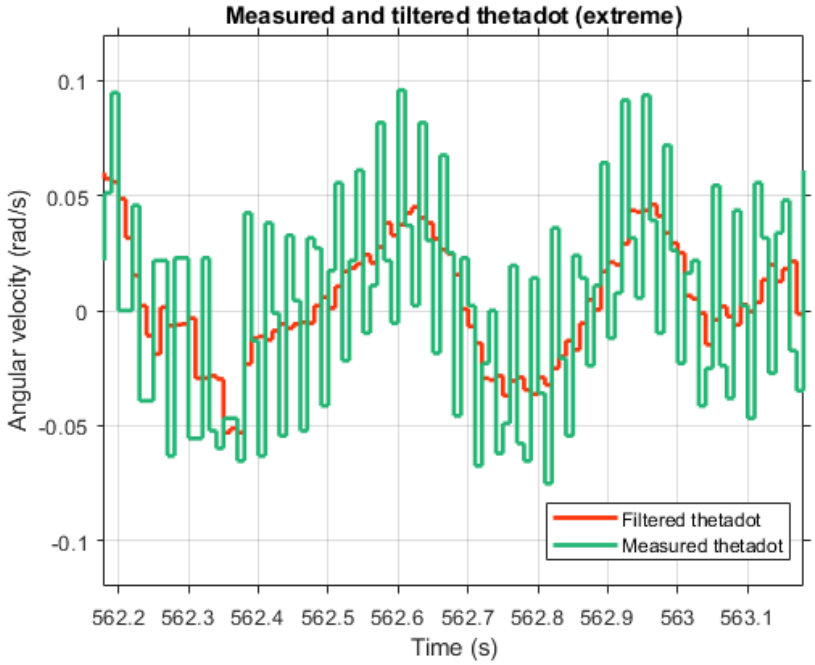

Figure 4: Filtering of $\dot{\theta}$, using a moving average of the last 3 values. During this period the bike was **not** in balance and was kept physically in place. It had an extreme flywheel speed of $\omega \approx -425$ rad/s. Also notice that magnitude of the y axis has tripled with respect to Figure 3.

5 PD(P) Controller

Let's start with a feedback controller (PD Controller), where we considered the output to be the angular position $y(t) = \theta(t)$, just like discussed in Section 3. The generic input equation of a PID controller is

$$u(t) = K_p(y(t) - y_{ref}(t)) + K_i \int_0^t (y(\tau) - y_{ref}(\tau))d\tau + K_d \frac{d}{dt}(y(t) - y_{ref}(t)).$$

The reference output value is taken to be $y_{ref}(t) = 0$ and no integral term is used in the control input resulting in

$$u(t) = K_p\theta(t) + K_d\dot{\theta}(t). \tag{3}$$

However, in practice this input for the fly wheel motor only keeps the bike upright for a couple of seconds. In Figure 5 the 3 state space variables (θ, $\dot{\theta}$ and ω) can be seen over time. Here the bike is briefly in balance but meanwhile the flywheel's angular velocity increases linearly. This is because when the measured angle $\theta = 0$ the bike is not necessarily exactly in balance, i.e. there is a bias. Thus requiring a constant acceleration of the fly wheel to keep the bike in balance. This is problematic as the flywheel has a maximum velocity at which point the DC motor can no longer generate a torque in the direction of the velocity.[3] (DC motors have the characteristic that their torque decreases linearly as the angular velocity increases, see Figure 13 in Appendix 9.2.)

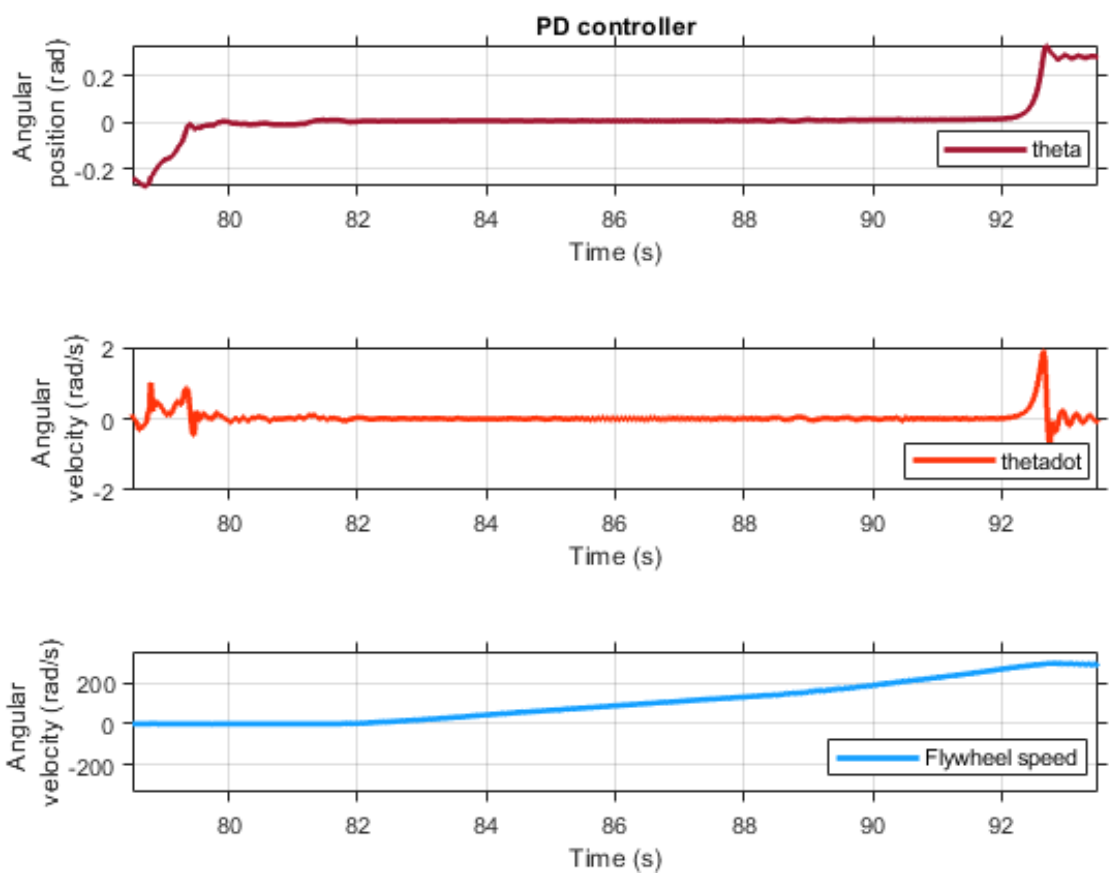

Figure 5: A briefly balancing bike using the PD controller (3), where the angular velocity grows linearly until the generated amount of torque becomes too small to keep it in balance and it falls. The bike has been moved (by hand) close to the equilibrium point to start the balancing.

The unbounded angular velocity ω of the flywheel can be mitigated by introducing ω into the controller function (making it a PDP controller).[3]

$$u(t) = K_p\theta(t) + K_d\dot{\theta}(t) + K_\omega\omega(t) \tag{4}$$

In Figure 6 the states of the system can be seen along with the "bounded" ω and observe that the system remains in balance. For the sake of making an easy comparison, the same y axis has been used in Figure 5 and 6.

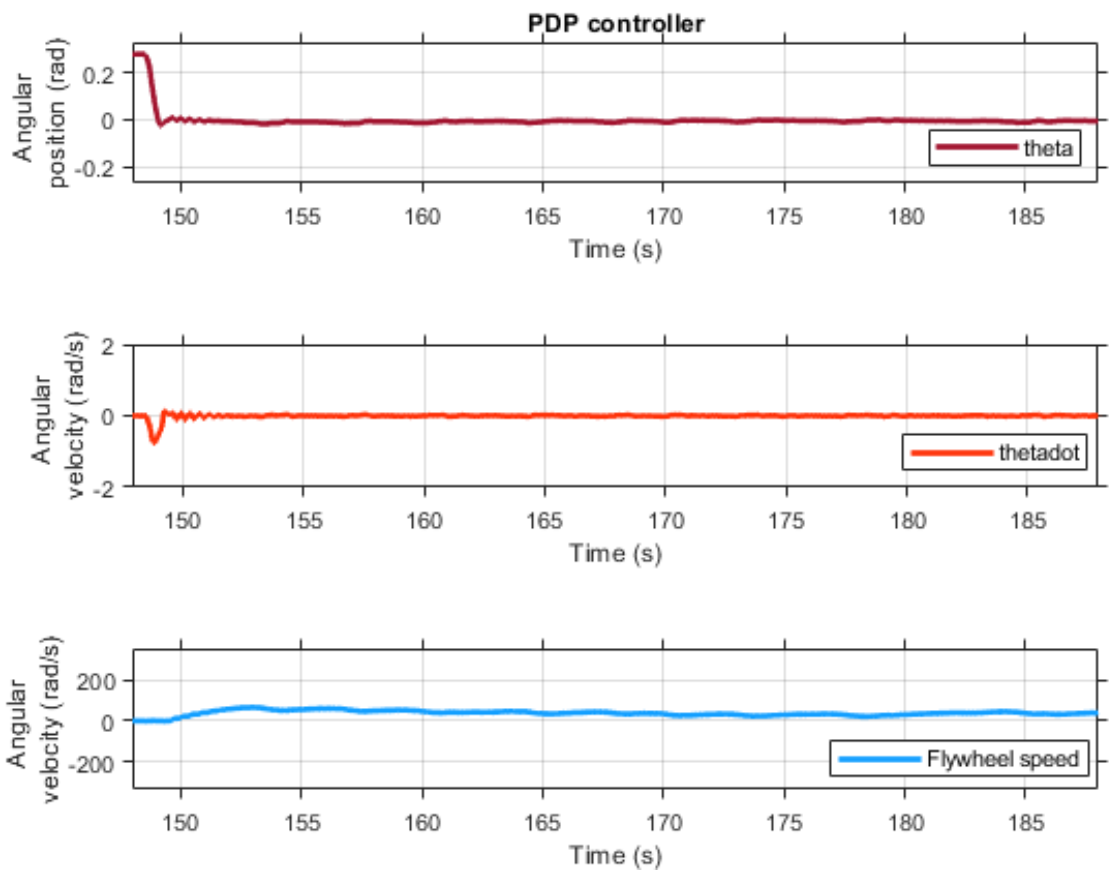

Figure 6: A balancing bike using the PDP controller (4), where the angular velocity becomes almost constant with respect to Figure 5. Also note that this remains in balance after it has been moved (by hand) close to the equilibrium point.

In Figure 7 the individual components of $u(t)$, namely $K_p\theta(t)$, $K_d\dot{\theta}(t)$ and $K_\omega\omega(t)$ are plotted over time to show each components' contribution in keeping the bike in balance. Observe that $K_\omega\omega(t)$ acts as a dynamic bias to counteract the term $K_p\theta(t)$. The data shown comes from the same test that generated Figure 6.

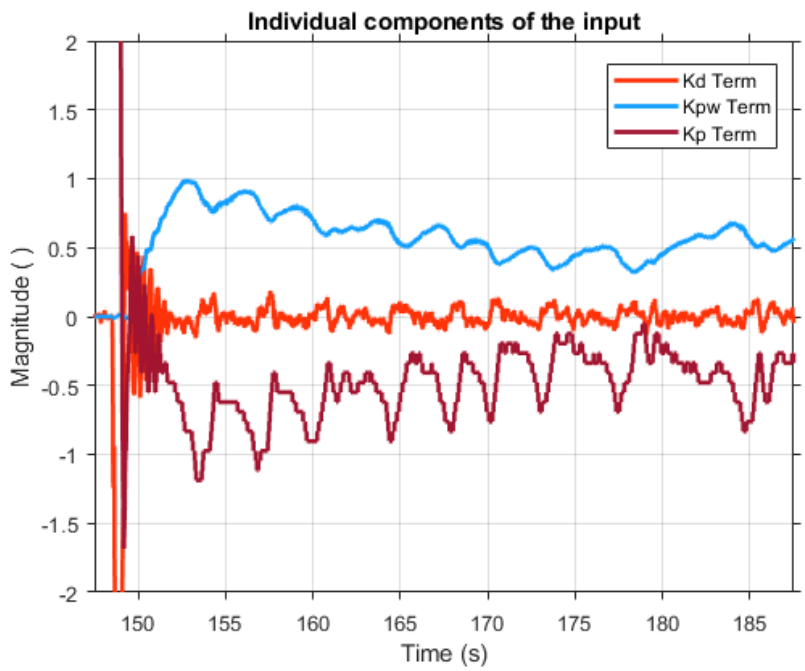

Figure 7: The individual components of $u(t)$, namely $K_p\theta(t)$, $K_d\dot{\theta}(t)$ and $K_\omega\omega(t)$ plotted over time. The data shown comes from the same test that generated Figure 6 and has the same colour legend.

The parameters used in the test that generated the graphs shown in this section can be found in Appendix 9.3.

6 Funnel Controller

The main goal was to apply the funnel to the self balancing bike, we will be attaching a Simulink file as well and a tutorial on how we constructed the funnel in Simulink and the foresight behind it, the tutorial can be removed or kept by the professor to his liking.
We will first look into the used equations used for the relative degree 2 funnel.

6.1 Degree 2 Funnel

In principle the Funnel controller works similar to the PID but the gains are adaptively bounded. The control objective of the Funnel controller is to track the error:

$$e = y - y_{ref} \tag{5}$$

which in our case is the θ.
and its derivative:

$$\dot{e} = \dot{y} - \dot{y}_{ref} \tag{6}$$

which is $\dot{\theta}$.
The terms y_{ref} & $\dot{y}_{ref}$ are set to zero which means that we want our self balancing bike to be upright with minimal movement of the flywheel.

The control is achieved by the funnel controller

$$u(t) = -K_0(t)^2 e(t) - k_1(t)\dot{e}(t). \tag{7}$$

Now we look at the respective funnel boundaries for e & $\dot{e}$,

$$k_0 = \frac{\varphi(t)}{1 - \varphi(t)|e(t)|}$$

and

$$k_1 = \frac{\varphi(t)}{1 - \varphi(t)|\dot{e}(t)|}.$$

These are the two high upper gain boundaries for e & $\dot{e}$. Displaying one for simplicity below.

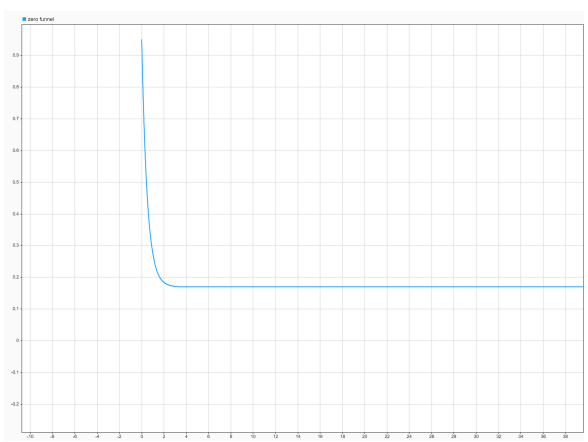

Figure 8: Upper boundary for theta for continuity the second graph also has the upper boundary.

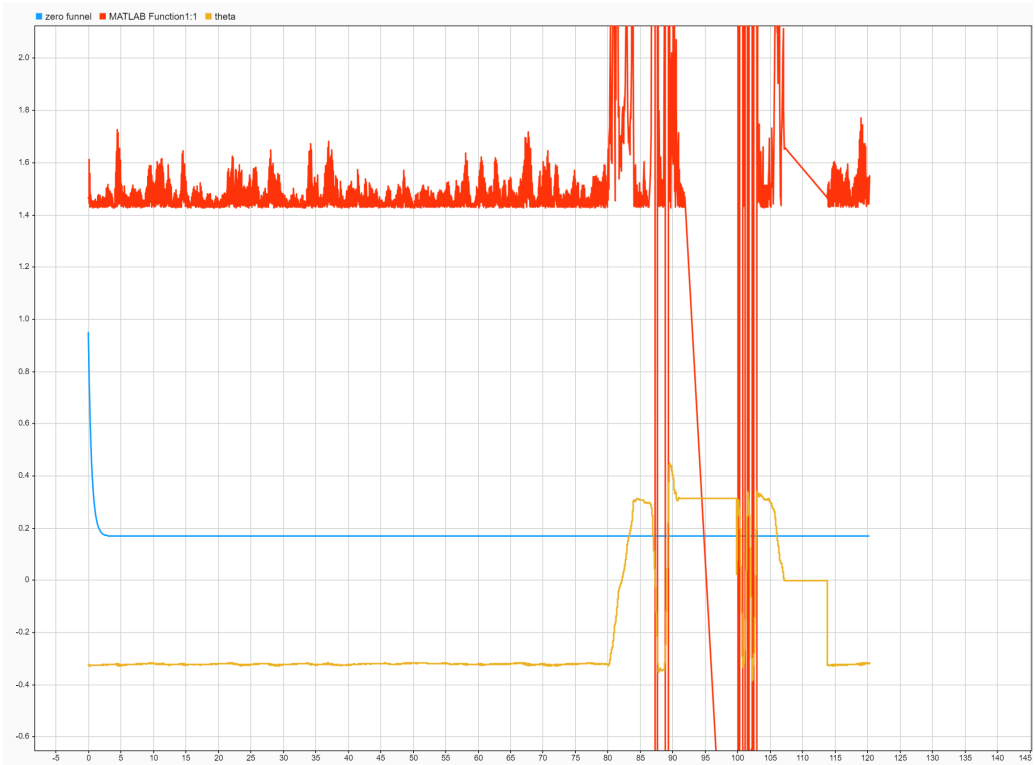

Figure 9: This is the gain for K_0 with respect to the values of theta and its funnel boundary.

We shall also look at the respective high gain with relation to the upper boundary again using the parameters of θ and its respective adaptive gain. One thing to note is that as θ arrives in close proximity to the boundary it can be seen in Figure 9 that the gain has now jumped to very large negative numbers.[1] [4]

What this means is that when these values approach the boundary the corresponding gain rises inversely to counteract the system to remain in the boundary giving the flywheel an ever increasing input as seen in Figure 10.

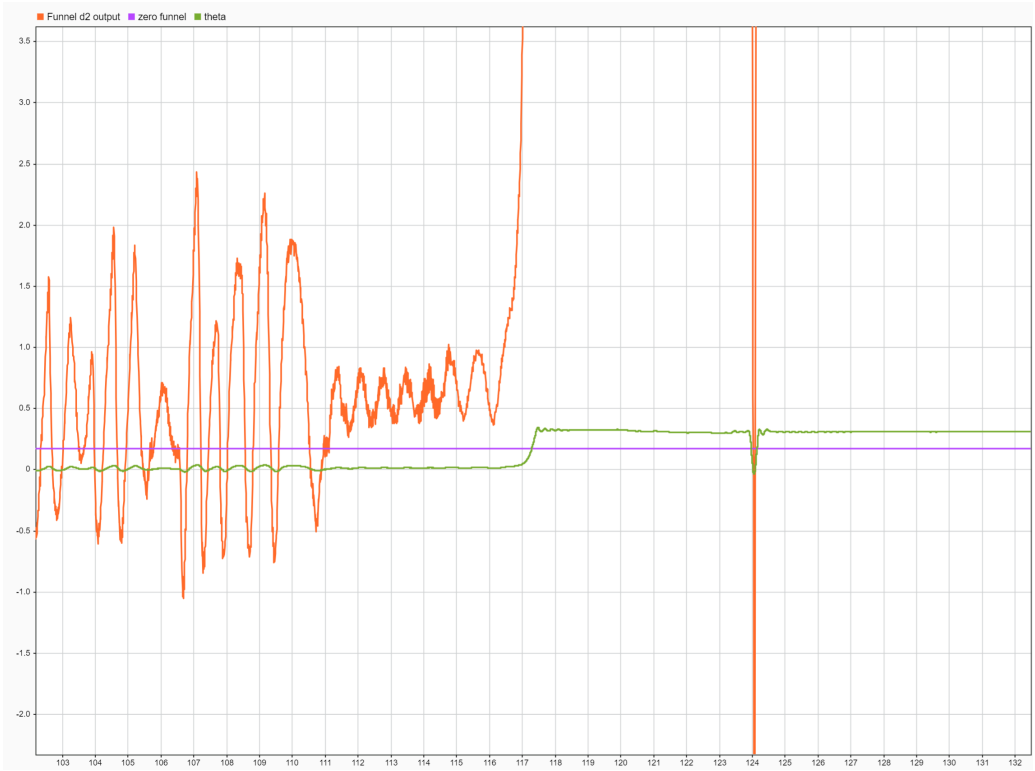

Figure 10: The Funnel output in orange with theta in green and funnel upper boundary purple this graph.

The Figure illustrates that as θ approaches the boundary there must be a maximal input the system can adhere to, hence there must be a input saturation applied.

6.2 Tuning the Funnel boundaries

The minimum value for the Funnel boundaries that we measured were $+/-0.16$ as seen in Figure 10 where the purple line dictates the upper boundary. Though initially it may seem like the boundary is not near the θ the system is perturbing from zero we see that the funnel output is quite large when compared to both the other values. We first started off with keeping the boundary as large as possible 0.4- then transitioned to lower the boundaries up until 0.16. at a lower boundary like 0.1 the system then becomes too sensitive to perturbations and can be see in Figure 11. When Figure 11 is compared to 10 it is easily be observed that changing the funnel boundary from 0.16 to 0.1 the funnel will now be unstable. One thing to note is that the scale is larger in Figure 11 when compared to 10.

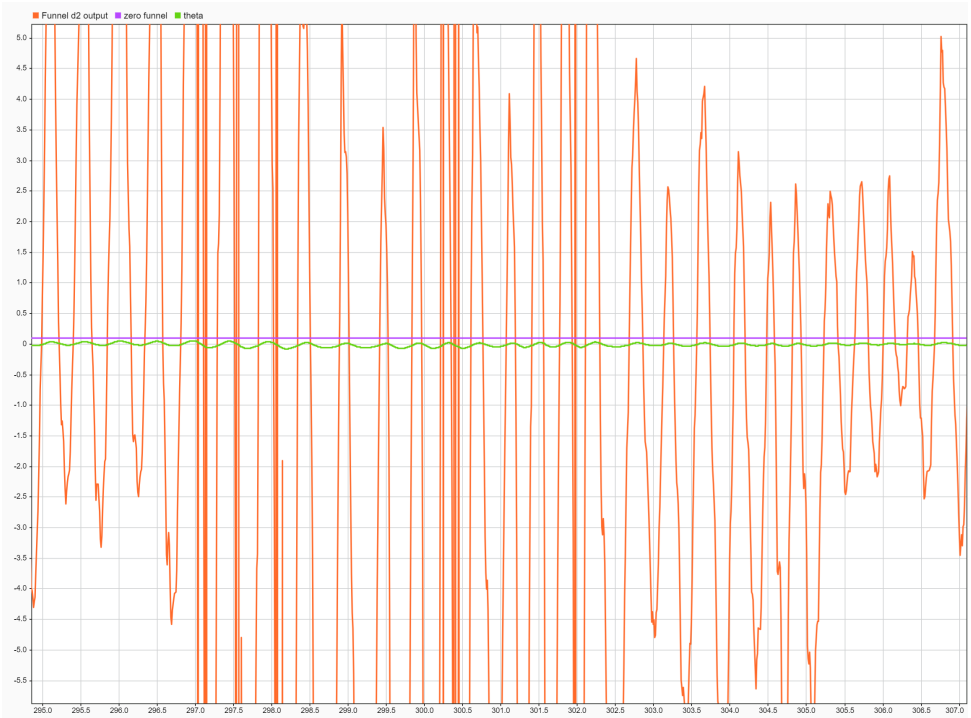

Figure 11: Funnel comparison with much lower boundary closer to theta

6.3 Simulink application of the Funnel

The application to Simulink in hindsight was fairly simple we will be providing the exact techniques in the appendix. The structure of the funnel will be discussed and the exact design will be placed in the appendix.
We wanted to combine both funnel and PDP control together using a parallel structure, and the result we obtained was an unstable system where the funnel controller and PDP control were working in parallel to balance the system. It can be seen that very small perturbations can cause the system to destabilize after many seconds of unstable balancing.

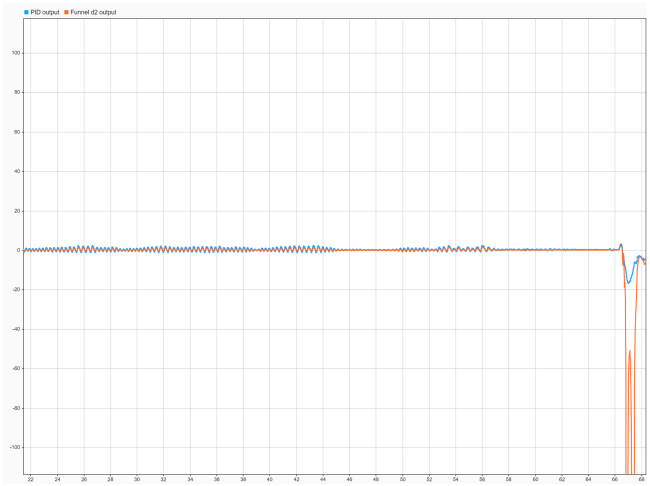

Figure 12: Funnel in orange and PID in blue.

6.4 Problems faced with the funnel

The issues posed with the funnel were:

- The proposed funnel must be of relative degrees higher than 2 supposedly 4. Where we consider the dynamics of the motor, i.e. from the electrical signal to inertia.
- Bike motor limitations with funnel: The adaptive gain are insufficient for output so the limitations lie within the ability of the motor to balance for high gains.

7 Conclusion

This implementation of the funnel controller for the self-balancing motorcycle demonstrated the system's ability to maintain upright under small disturbances for a small time frame then it would destabilize. Using the controller the tilt angle of the bike was regulated through the torque on the flywheel making the motorcycle balance. Though the theory says the funnel control works better than the classical controller and the ability to tolerate larger disturbances are better for the funnel, we were not able to establish this using the experimental results. Practical limitations such as noise, uncertainty in the relative degree may have contributed to the gap between this theoretical and observed performance. Nevertheless, the funnel controller provided a stable and adaptive response without requiring a model of the system, making its potential for control applications in nonlinear systems.

8 Future Scope

- Implementing a higher relative degree due to motor dynamics will be necessary in order for the funnel to bound the output with the existing funnels.

- Better understanding of input saturation and understanding of the system dynamics will be a must in order to continue applying the funnel for future projects.

- For the application of model Reference control it can be implemented for relatively better control of the bike. This reference can also be applied to the funnel controller so as opposed to using a constant block 0 for both e and $\dot{e}$.

- Instead of PDP, using a PID the **I** term will remove the need to change the bias for every bike and surface. If this responds fast enough to changes in the bias it can be used to keep the bike in balance when taking turns at speeds where the centrifugal force becomes significant.

References

[1] J. Hu, S. Trenn, X. Zhu. *A novel two stages funnel controller limiting the error derivative.* Systems & Control Letters, vol. 179, pp. 105601, 2023.

[2] C.M. Hackl. *Non-Identifier Based Adaptive Control in Mechatronics–Theory and Application.* Lecture Notes in Control and Information Sciences, vol. 466, Springer-Verlag, Cham, Switzerland, 2017. http://dx.doi.org/10.1007/978-3-319-55036-7.

[3] Arduino Education. Arduino Engineering Kit Rev2. Retrieved April 16, 2025. https://edu-content-preview.arduino.cc/content-preview/university/project/CONTENTPREVIEW+AEKR2

[4] Hackl, C.M., Hopfe, N., Ilchmann, A., Mueller, M., & Trenn, S. *Funnel control for systems with relative degree two.* SIAM Journal on Control and Optimization, vol. 51, no. 2, pp. 965–995, 2013.

9 Appendix

9.1 Moments of inertia

For both the moments of inertia the parallel axis theorem is used. The moment of inertia of the pendulum (rod), a tall cylinder, around point A is

$$I_r^A = I_r^B + m_r l_{AB}^2 = \frac{1}{12} m_r (3r^2 + l_{AD}^2) + m_r l_{AB}^2,$$

where m_r is the mass of the pendulum,
r is the radius of the pendulum and
l_{AB} is the distance between points A and B.

The moment of inertia of the flywheel around point A is

$$I_f^A = I_f^C + m_f l_{AC}^2 = \frac{1}{2} m_f R^2 + m_f l_{AC}^2,$$

where m_f is the mass of the flywheel,
R is the radius of the flywheel and
l_{AC} is the distance between points A and C.

Substituting these moments of inertia into (1) we obtain

$$\dot{x}(t) = \begin{bmatrix} \dot{\theta} \\ \ddot{\theta} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} \dot{\theta} \\ \frac{g \sin(\theta)(m_r l_{AB} + m_f l_{AC}) + u(t)}{m_f (0.5R^2 + l_{AC}^2) + \frac{1}{12} m_r (3r^2 + l_{AD}^2) + m_r l_{AB}^2} \\ \frac{u(t)}{I_f^C} - \ddot{\theta} \end{bmatrix}$$

9.2 DC motor torque vs rotational speed

The torque that a DC motor can generate decreases as the rotational speed increases, see Figure 13.

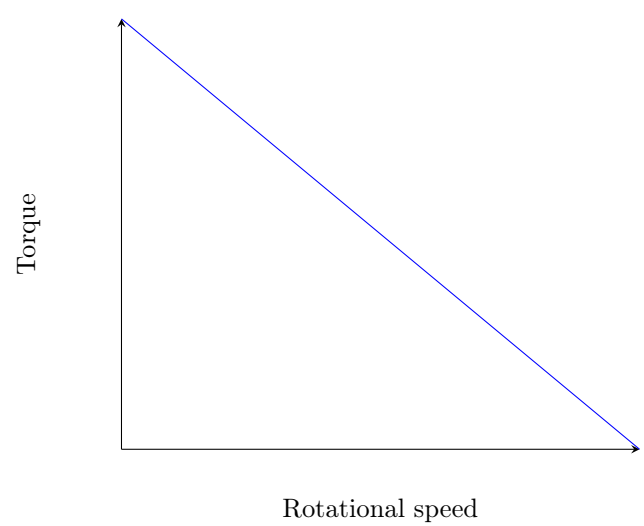

Figure 13: Torque vs rotational speed in a DC motor.

9.3 Constants used in PD(P) controller

In Table 1, the constants that have been used to generate the data can be seen.

Parameter	Value
θ_{bias}	-0.015
K_p	65
K_d	5
K_ω	0.015

Table 1: The constants that are used in the test with the filter and PD(P) controller.

9.4 Controller and Plant

We start with our design of the controller and system. A bus is used to connect the digital controller and the motorcycle.

Figure 14: Controller and Motorcycle structure.

This is the structure of the Controller an Motorcycle which shows inputs from the motorcycle flow into the digital controller and the controller then sends an output to the motors on the motorcycle.

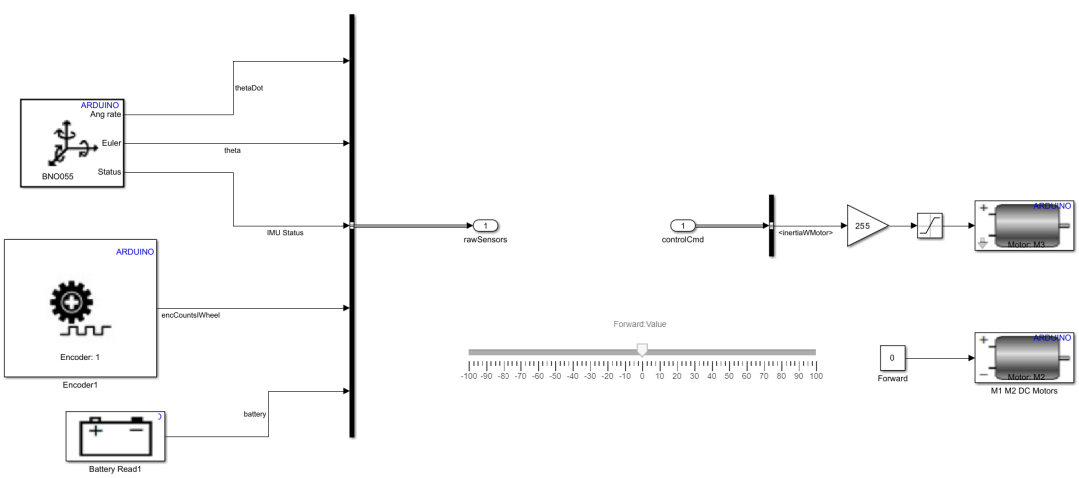

Figure 15: Smaller components connection to bus as input to the controller.

On the left side of the image we can see that the input of the motorcycle sends the information of 5 variables top to bottom: angular rate $\dot{\theta}$ Euler which is θ and status which tells us whether the IMU is calibrated or not. The 4th block is the encoder, which counts the number of rotations of the flywheel which is needed, the 5th is Battery read tells us how much voltage the battery is able to send to the system. On the right-hand side we have 2 motors on the top motor M3 is the flywheel and it is connected to the digital controller it has a saturation of 255 the maximum number the system can handle and then it is connected via saturation to prevent any circuit

malfunction due to high voltage. Lastly we have the M2 motor which is designed to move the bike back and forth.

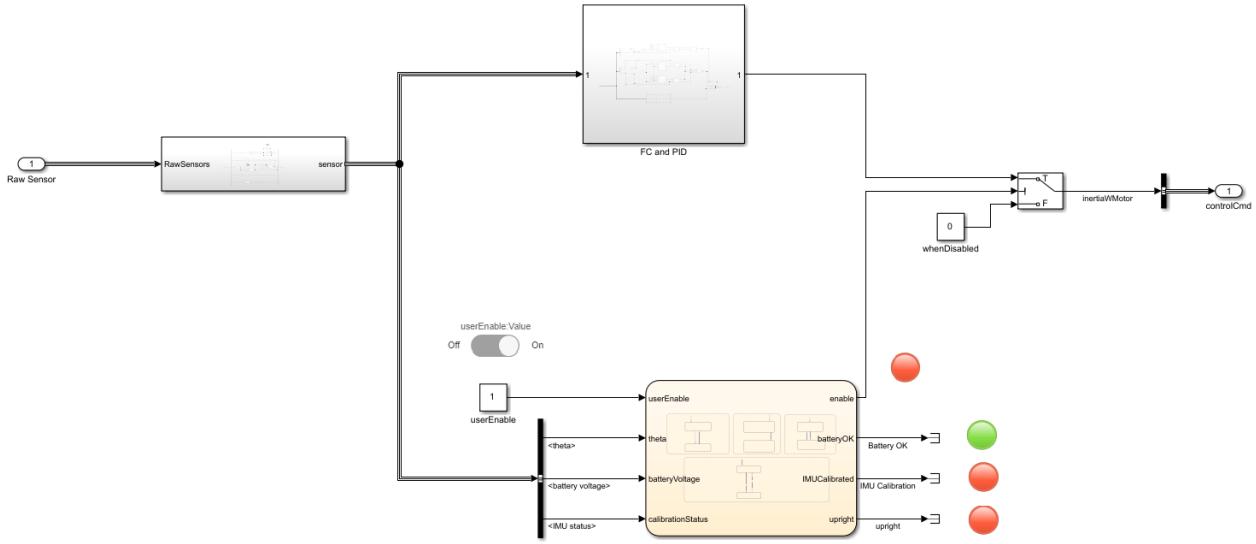

Figure 16: This Figure is the overall design of the logic board sensor data connection and controller.

This Figure shows how our digital controller is connected to a switch via a chart on the bottom and it defines 4 stats:

- Battery OK
- IMU Calibration
- Upright

When all the values are at a certain threshold we will see that all the lights turn green which switches the output from 0 to the output of the controller.

9.5 Internal structure of Funnel controller and PID

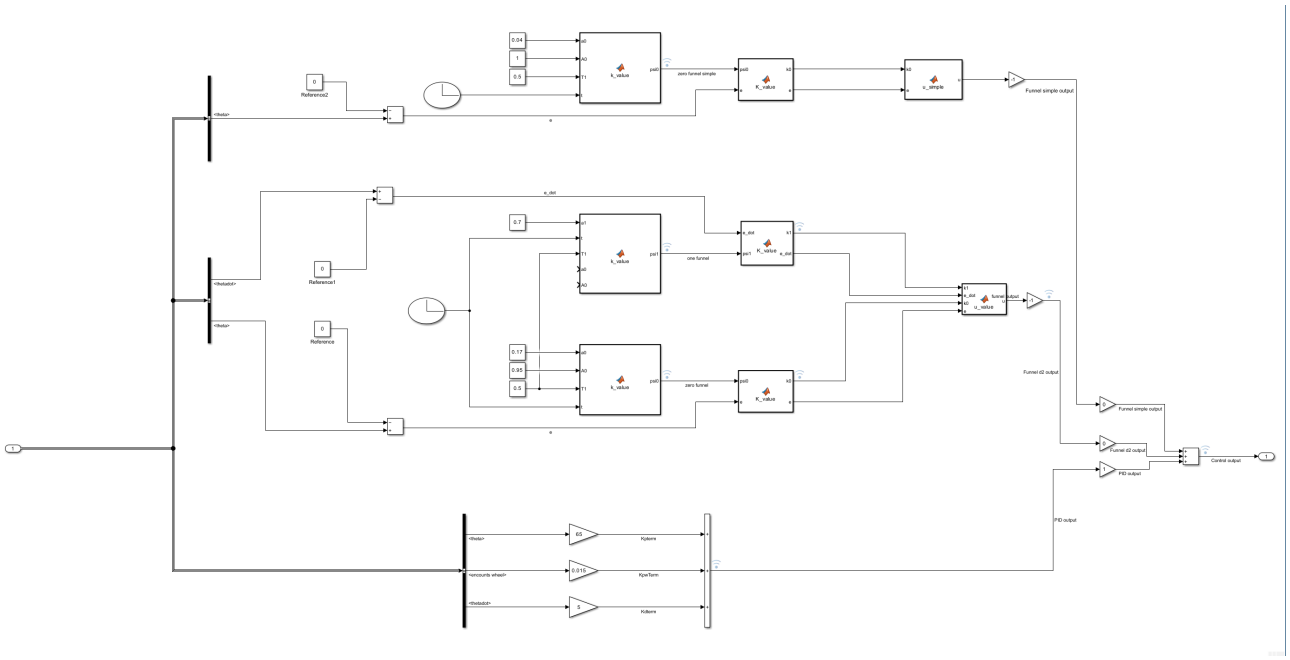

Figure 17: This Figure displays our over all structure of the controllers that we have implemented on the Self balancing bike.

The internal structure from top to bottom has the following:

- Relative Degree 1 funnel
- Relative Degree 2 funnel
- PDP controller

The following is a list of all the blocks used be aware that this is the list for the version 2024b it may be subjected to change in the preceding versions.The design for the Simulink model for PID controller can be found on [3],this may also be subjected to change for the self balancing motorcycle please look accordingly

- Clock Block
- Sum Block
- Gain Block
- Bus Selector
- Matlab Function Block
- Chart Block for Logic
- Constant blocks to be connected to various other blocks for Matlab functions and Summation blocks.
- Imu sensor BNO055, Encoder 1 and battery reader motor M1 M2 DC Motor